r2-vis trace methodology

Radiance 2 technical specification, document 4

r2-spec

Version 0.3, draft of 2026-10-21

Contents

1 Introduction **3**

1.1 Scope . . . 3

1.2 Conventions . . . 3

1.3 Terms . . . 4

2 Principles **4**

2.1 The trace observes the run . . . 4

2.2 Two stages . . . 4

2.3 Completeness . . . 5

2.4 One processed trace for every reader . . . 5

3 Raw dump **5**

3.1 Command line . . . 5

3.2 Files . . . 6

3.3 Manifest . . . 6

3.3.1 Resources . . . 7

3.3.2 Links, banks and channels . . . 7

3.3.3 Label schema . . . 8

3.4 Records . . . 8

3.4.1 Tasks . . . 8

3.4.2 Chunks and dependencies . . . 9

3.4.3 Charges . . . 9

3.4.4 Occupants . . . 9

3.4.5 Buses without a rate . . . 10

3.4.6 Programs, marks and table entries . . . 10

3.5 Partial dumps . . . 11
3.6 Summary . . . 11
3.7 Versions . . . 11

4 Post-processing **11**
4.1 Command line . . . 11
4.2 Stages . . . 12
4.3 Checks . . . 12
4.4 Traffic class . . . 13
4.5 Ceiling . . . 13
4.6 Time structures . . . 13
4.7 Compression and encoding . . . 14
4.8 Size and time . . . 14

5 Processed trace **15**
5.1 Manifest . . . 15
5.2 Tables . . . 15
5.3 SQL access . . . 15

6 Views **16**
6.1 Architecture . . . 16
6.2 Shared state . . . 17
6.3 Block diagram . . . 17
6.4 Timeline . . . 17
6.5 Attribution . . . 18
6.6 Instant view . . . 19
6.7 Presentation . . . 19

1 Introduction

This document specifies how a run of r2-sim becomes a trace, and how a person or a program reads that trace. It has three parts:

1. The **raw dump**: the files that r2-sim writes during a run (Section 3).
2. The **converter**, `r2t-convert`: it reads a raw dump once, checks it, derives the data that the readers need and writes a **processed trace** (Sections 4 and 5).
3. The views of the r2-vis graphical interface, which read a processed trace (Section 6).

Figure 1 shows the path of the data. r2-sim writes the raw dump. The converter writes the processed trace, a directory of Parquet tables. The r2-vis interface and SQL tools read the processed trace. The interface does not read the raw dump.

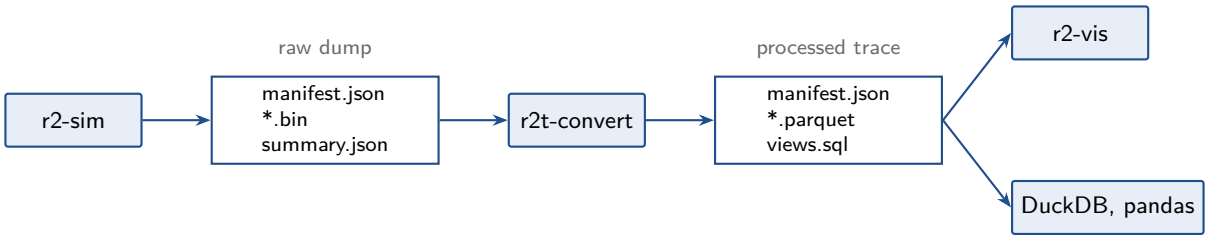


Figure 1: From a run to a view. Solid arrows: data files. The converter runs once per run.

1.1 Scope

The trace records what the simulator models:

- the lifetime of every task, from the invoke call to the retirement of its entry (Section 8);
- every chunk of every task, with its interval and its culprit resource;
- every dependency, with its producer;
- every **charge**: one occupation of one resource by a chunk or by one hop of a message, with its interval, its amount and its usage in slots (Section 5);
- the programs of every Neutrino, and the program marks;
- the entries of the Neutrino tables that the task records do not give (Section 6).

The planes, the resources, the slots and the protocols are defined in document 3 (Sections 2, 4 and 5) and in document 1. This document defines only how they appear in a trace.

1.2 Conventions

- Times are SM cycles of the global clock (Section 2). A time interval is written $[a, b)$. In this document, a *range* is always an address interval, as in the glossary.
- Sizes use IEC prefixes (KiB, MiB). Rates are units per SM cycle: bytes, MAC, lane operations or instructions.
- The raw dump has a version (`manifest.version`); the current version is 1.9. The processed trace has its own version; the current version is 1.8. Table 5 lists the versions.
- Field names, file names and keys are in `typewriter`. Message names are in sans serif.

1.3 Terms

This document uses the terms of the glossary (Section 4) with their meanings there. In particular, the *occupant* of a charge is the task or the program that the charge serves; the *owner* of a range is always a Neutrino.

2 Principles

The trace serves two kinds of readers: a person who looks at a run in the r2-vis interface, and a program (for example an agent) that queries a run in SQL. The trace must not change the run, and it must be cheap to make. The rules in this section follow from these needs.

2.1 The trace observes the run

Rule V2.1 (observation only). Tracing must not change the simulated execution. A run with tracing on and the same run with tracing off must end at the same cycle and must produce the same statistics.

Rule V2.2 (no cost when off). With tracing off, the cost of the tracer must be one branch per observation point. The simulator must not keep data for the trace when tracing is off.

Note. The performance target of r2-sim for tracing is at most 1.15 times the time of the untraced run. On one VLM layer of the forward pass, the measured cost is 1.05 times with only the table entries on, and 1.19 times with every record type on.

2.2 Two stages

The trace has two stages. The raw dump is cheap to write. The processed trace is cheap to read. The converter does all the work between them once.

Rule V2.3 (cheap to write). The raw dump must consist of fixed-size, little-endian binary records that the simulator appends in any order. The simulator must not sort records and must not compute data that the converter can derive.

Rule V2.4 (self-describing). Every raw dump must contain a manifest that gives the layout of every record file, the topology, the resources and the name tables. A reader must take the record layouts from the manifest, so that a later version can add fields at the end of a record.

Rule V2.5 (checked conversion). The converter must check every raw dump against the rules of Section 4.3 and must report each violation. A trace that fails a check is still written, so that the failure can be inspected.

Note. The two stages keep the simulator simple. A raw record is written in a few nanoseconds from data that the simulator has at hand. Sorting, joining, binning and compression happen once, in the converter, outside the run.

2.3 Completeness

Rule V2.6 (every charge). The raw dump must record every charge of every resource that carries traffic, also on a bus without a rate. For such a bus the tracer writes the charges that the simulator does not model (Section 3.4.5).

Rule V2.7 (every occupant). Every charge must name its occupant: a task, or a program for work that serves no task (Section 3.4.4).

Rule V2.8 (partial dumps). A raw dump can be restricted by design subtree, by record type and by time window. The manifest must record every restriction, and the topology tables must always describe the whole device (Section 3.5).

2.4 One processed trace for every reader

Rule V2.9 (columnar and flat). The processed trace must be a directory of Parquet tables with flat schemas (no nested columns), plus a manifest that describes every table and column in plain words, plus SQL views. Every reader (the interface, DuckDB, pandas, polars) reads the same files.

Rule V2.10 (exact values). Every value that a view shows must equal the value that a query over the charges gives for the same resource and time interval, within floating point rounding. Approximations (for example a value from bins while the charges load) must be marked.

Rule V2.11 (compatibility). The converter must read every older raw dump version. The interface must open every older processed trace version.

Note. Parquet gives what the interface needs from a custom format: columns that are compressed one by one, row groups with minimum and maximum statistics for time-interval pruning, and reads by byte range. It also gives SQL access without a separate export.

3 Raw dump

r2-sim writes the raw dump during a run. This section specifies raw dump version 1.9.

3.1 Command line

```
r2sim run ... --trace-raw DIR [--trace-zstd]
```

- `--trace-raw DIR` writes the files of Table 1 into `DIR`.
- `--trace-zstd` compresses each binary file as one zstd stream (level 1 to 3) and adds `.zst` to its name.
- The restrictions of a partial dump (Section 3.5) come from `--trace-config FILE` or `--set trace.KEY=VALUE`.

File	Written	Content
`manifest.json`	before the first event	run identity, filters, record layouts, topology, resources, name tables
`strings.txt`	appended	program and mark names, one per line
`tasks.bin`	appended	one record per task (112 B)
`chunks.bin`	appended	one record per chunk (32 B)
`deps.bin`	appended	one record per dependency (24 B)
`charges.bin`	appended	one record per charge and direction (32 B)
`programs.bin`	appended	one record per program run (32 B)
`marks.bin`	appended	one record per program mark (16 B)
`entries.bin`	appended	one record per Neutrino table entry (48 B)
`summary.json`	at the end	end cycle, counts, fatal error, phases

Table 1: The files of a raw dump.

3.2 Files

Rule V3.1 (record files). A record file is a sequence of records of one fixed size, with no header. All values are little-endian. Fields are at the offsets of the layout, with no padding other than fields named `pad`, which are zero. Null is the maximum value of the field type (all bits one). Times are `u64` SM cycles.

Rule V3.2 (order). The records of a file can be in any order. The simulator writes a record when its last field is known, for example a task at its retirement.

Rule V3.3 (fatal end). On a fatal error (a deadlock included), the simulator must flush every file, write the complete records and write `summary.json` with the error. A reader must accept a dump without `summary.json` and must drop a partial last record.

A string id is the 0-based line number in `strings.txt`. The simulator writes each string once, when it first uses it.

3.3 Manifest

The manifest is one JSON object. Its keys:

- `format` (`"r2raw"`) and `version` (`[1, 7]`).
- `run`: `config`, `program`, `params`, the model flags in `overrides` (for example `"--acc-buffer 1048576"`), `sim_commit` (with the suffix `-dirty` for a tree with uncommitted changes), `clock_ghz`, `granule` and `created` (RFC 3339, UTC).
- `filters`: the restrictions of a partial dump (Section 3.5).
- `records`: the layout of every record file, a list of `[name, type]` pairs in file order, with the types `u8`, `u16`, `u32` and `u64`.
- the topology tables of Table 2.
- the name tables `opcodes` and `message_kinds`, and the label schema (Section 3.3.3).

Every entry of a topology table has an `id` equal to its index in its list. The ids are the ids of the simulator: `nu` is the Neutrino id, `pe` the PE id and `resource` the resource id.

Table	Fields
`nodes`	`path` (`root.gpc0.sm1`), `parent`, and `neutrino`: the capacity of each Neutrino table
`pes`	`path`, `node`, `type` (`tensor`, `vector`, `dma`)
`buses`	`path`, `node`, `topology`, `control`, `bw_bytes` (null if the bus has no rate), `wr_bytes`, `latency`
`attachments`	`bus`, `name`, `component`, `component_type`, `role`, the port resources `in`, `out` and the write channels `in_wr`, `out_wr`
`links`	`path`, `node`, `control`, `from_bus`, `to_bus`, the end attachments, the direction resources `ab`, `ba` (and `ab_wr`, `ba_wr`), `bw_bytes`, `wr_bytes`, `beat_bytes`, `latency`, `cache`, `hash`
`memories`	`path`, `node`, `type` (`scratch`, `dram`, `cache`, `sync`), `ranges`, `attachments`, `port`, `wport`, `bw_bytes`, `wr_bytes`, `latency`
`resources`	`name`, `kind`, `rate`, `capacity`, `clock`, `rates`, `beat_bytes`, `owner`, `role`

Table 2: The topology tables of the manifest.

3.3.1 Resources

A resource entry gives the values that a reader needs to compute a utilization:

- `rate`: units per SM cycle, as `[num, den]`; null for a bus without a rate.
- `capacity`: slots per SM cycle, as `[num, den]`. For a bandwidth resource the capacity is *rate*/*beat_bytes*; this equals *bw_bytes*/*beat_bytes* × *clock*, because `rate` is already per SM cycle. A compute PE has one slot per cycle of its clock.
- `clock`: the clock ratio of the resource, default `[1, 1]`.
- `rates`: for a compute resource whose rate depends on the operand type, the list of its rate classes, for example `{"name": "bf16", "rate": [8192, 1]}`. The field `rate` is then the largest rate. A mixed-type operation runs at the rate class of its slower operand.
- `role`: what the resource is in the topology: `port` (of an attachment, with a direction `in` or `out`), `bus`, `link` (with a direction `ab` or `ba`), `mem_port`, `pe`, `scalar` or `complete` (of a Neutrino). A role has `"write": true` for a write port or a write channel.

Example. In the two-bank configuration of the 2p device, `root.gpc0.sm1.smem.b0.port` has the rate 128 B/cycle, the beat 64 B and the capacity 2. `root.gpc0.sm1.atlas.array` has the capacity 1 and the rate classes bf16 (8192 MAC/cycle), fp8 (16384) and mxfp4 (32768).

3.3.2 Links, banks and channels

A link joins two buses (Section 3.4). Its direction resources are named `<link>.ab` and `<link>.ba`, where `ab` goes from `from_bus` to `to_bus`. The manifest does not store up and down: the converter derives them from the nodes of the two buses.

A memory with banks is one memory entry per bank, named `<memory>.b<i>`, each with its own ports. A **hash link** in front of banked memory has a `hash` object: the linear address interval, the target interval, the index masks (bank bit j is the parity of $address \,\&\, index_j$), the dropped bits, the granule and the slice size. Records keep linear addresses; the bank of a transfer shows only through the resource that it charges.

A bus or a link with `wr_bytes` has separate read and write channels. The write channels are

their own resources, named `<name>.wr`, with the same beat as the read channel. A memory with `wport` has a separate write port. The charges of a write use the write resources.

3.3.3 Label schema

The label of a descriptor is the software identity of a task. The manifest describes its bit fields, so that readers have no copy of the encoding:

```
"label": {"fields": [{"name": "kind", "shift": 56, "bits": 8,
                      "enum": ["none", "router", "qkv", ...]},
                     {"name": "step", "shift": 48, "bits": 8},
                     {"name": "layer", "shift": 40, "bits": 8},
                     {"name": "unit", "shift": 32, "bits": 8},
                     {"name": "idx", "shift": 0, "bits": 32}],
          "format": "{kind} s{step} l{layer} u{unit} #{idx}"}
```

A field name becomes a column name of the processed trace, so it must not be an SQL keyword.

3.4 Records

3.4.1 Tasks

Offset	Field	Type	Meaning
0	`gid`	u32	task id, in invoke order
4	`nu`	u16	the Neutrino that tracks the task (the owner of the destination range)
6	`src_nu`	u16	the Neutrino whose program called the invoke
8	`pe`	u16	the PE; null for `join` and `spawn`
10	`opcode`	u16	index into `opcodes`
12	`version`	u32	version of a versioned destination range
16	`label`	u64	descriptor label
24	`dst_base`	u64	destination range base, in bytes
32	`dst_len`	u64	destination span in bytes: the bound times the granule
40	`call`	u64	the program issued the invoke
48	`invoke`	u64	entry into the scoreboard (for a forwarded invoke: arrival)
56	`ready`	u64	the last dependency resolved
64	`issue`	u64	entry into the execution table
72	`complete`	u64	completion reached the tables
80	`retire`	u64	the entry left the tables
88	`queued`	u64	forwarded invoke: entry into the outbound invoke table
96	`sent`	u64	forwarded invoke: the Invoke left that table
104	`flags`	u32	bit 0: untracked task
108	`dst_mask`	u32	destination mask, in granules

Table 3: A task record (112 B). A time that did not occur is null.

The lifecycle times are those of Section 8. The difference *invoke* − *call* is the time that an invoke spends in the program and on the fabric before the owner holds it.

The destination range has the three fields of Section 2.3 in granules of `run.granule`: the base is *dst_base*/*granule*, the bound is *dst_len*/*granule* and the mask is `dst_mask`. Granule *base* + *i*,

with $0 \le i < bound$, belongs to the range when $i \,\&\, \neg mask = 0$. A plain range has the mask all ones.

An untracked task (Section 3.1) has `version`, `queued` and `sent` null, and no dependency names it as its producer. A fence (Rule N3.11) has no record: its Neutrino work is a charge of its program.

3.4.2 Chunks and dependencies

A chunk record (32 B) has the fields `gid`, `index` (in request order), `start`, `end`, `culprit` and `queue_culprit`. The **culprit** is the resource with the longest own duration in the chunk; the queue culprit is the resource whose earliest start delayed the chunk most (Section 5).

A dependency record (24 B) has `consumer`, `producer`, `resolved`, `index`, `state` and `flags`. The producer is the task whose destination range the dependency resolved to, also when that task has already retired. The state is the producer's phase at the consumer's invoke: 0 not invoked, 1 in the scoreboard, 2 executing, 3 done. Bit 0 of `flags` marks a dependency on a range of another Neutrino; bit 1 marks a versioned dependency.

3.4.3 Charges

A charge record holds one direction of one charge (Figure 2). A charge with read and write parts gives two records with the same `start` and `span`.

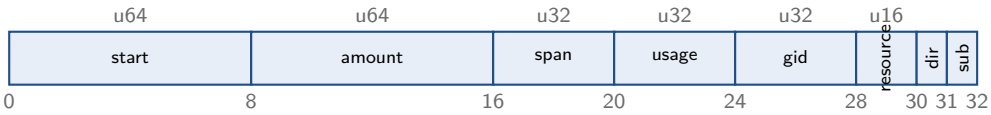


Figure 2: A charge record, 32 B. Numbers are byte offsets.

- `amount`: the units of this direction (MAC, lane operations, bytes or instructions).
- `span`: the cycles of the whole charge. The charge occupies $[start, start + span)$.
- `usage`: the slots that this direction holds, multiplied by 65536 and rounded down. The records of one charge sum to the usage of the charge. Its slot-cycles are $usage/65536 \times span$.
- `gid`: the occupant (Section 3.4.4).
- `dir`: 0 read, 1 write, 2 work.
- `sub`: for a protocol message, the message kind; for a charge on a resource with rate classes, the rate class it ran at; null otherwise.

Rule V3.4 (capacity). On a resource with a rate, at every cycle the sum of `usage` over the charges active at that cycle must be at most $\lfloor capacity \times 65536 \rfloor$.

Note. A narrow beat holds a whole slot. So the slot usage of a resource and its byte throughput can differ: a 4 B master on a link with 32 B beats holds one slot and moves 4 B per cycle.

3.4.4 Occupants

The field `gid` names the occupant of a charge. A task occupant is the task gid. A program occupant is `0x80000000` | *spawn*, where `spawn` is the gid of the task that started the program; the root program is `0xFFFFFFFE`.

Charge	Occupant
a chunk of a task	the task
Invoke, InvokeAck	the invoked task
Await, AwaitAck, Resolve	the consumer task, or the waiting program
Start, Return	the `spawn` task
VersionReset	the sending program
Neutrino work of an invoke call	the invoked task
other Neutrino work, loads and stores	the program of that Neutrino

Table 4: The occupant of a charge.

The gid of a forwarded task exists only from its invoke at the owner. The tracer holds the charges of an Invoke and of its invoke call until then.

3.4.5 Buses without a rate

A bus without a rate costs no simulation time, so the simulator has no resource for it. The tracer lists its ports as resources with a null rate and ids above the other resources, and writes their charges:

- For a transfer chunk, one charge per port of the route, over the interval of the chunk. The ports from the source to the engine have direction read; the ports from the engine to the destination have direction write.
- For a message, one charge per port, over the interval of the rated hop that the simulator folds the bus into.
- Each such charge has the usage of one notional slot and a span of at least 1 cycle. Charges on such a port can overlap without limit.

3.4.6 Programs, marks and table entries

A program record (32 B) has `nu`, `name` (a string id), `spawn`, `start` and `end`. A mark record (16 B) has `time`, `name` and `nu`.

An entry record (48 B) holds one entry of a Neutrino table that the task records do not give: the pending table, the version table, the inbound reference table, the inbound invoke table, the outbound dependency table and the outbound invoke table (Section 6). It has `start`, `end`, the key or range (`base`, `len`, `version`), `gid`, `nu`, `peer`, `table` and `flags`. An entry that holds a task's range has the task's `gid`; the mask of the range is that of the task. A forwarded VersionReset takes an entry of the inbound invoke table, without a task, until it takes effect at the head of the table (Rule N3.8). The scoreboard and the execution table come from the task records (`invoke`, `issue`, `retire`). The occupancy of a table at cycle t is the number of its entries with $start \leq t < end$.

Example. An inbound reference entry at the owner and the outbound dependency entry at the requester of the same Await have the same `gid`: the consumer task. A reader can join the two sides of one remote dependency.

3.5 Partial dumps

The manifest key `filters` records the restrictions of a dump: `nodes` (subtree paths), `records` (the files written), `types` (the record types kept), `window` (the recorded window $[lo, hi)$) and `partial`.

- A task is kept if $[invoke, retire)$ overlaps the window and its `nu` is in a listed subtree.
- A chunk, a charge, a program or an entry is kept if its interval overlaps the window. A charge is kept if the owner node of its resource is in a listed subtree.
- A dependency is kept if its consumer is kept.
- In a partial dump, a gid can name a task without a record. The converter keeps the reference and marks it as outside the trace.

3.6 Summary

`summary.json` has the end cycle, the event count, the wall time, the record counts, the fatal error (or null) and the phases of the program.

3.7 Versions

Version	Change
1.0	the files of Table 1 without `entries.bin`
1.1	integer lanes per resource
1.2	the `links` table replaces `bridges`
1.3	`entries.bin`; Neutrino table capacities in `nodes`
1.4	banks, hash links, memory write ports
1.5	read and write channels
1.6	an occupant on every charge
1.7	fractional slots (`capacity`, `clock`, `usage`) and rate classes
1.8	inbound invoke entries for VersionReset; `flags` in the task record (untracked tasks); task record 112 B
1.9	`dst_mask` in the task record; destination ranges in whole granules

Table 5: The raw dump versions. The converter reads all of them (Rule V2.11).

4 Post-processing

The converter reads a raw dump and writes a processed trace. It runs once per run, on the host that ran the simulator.

4.1 Command line

```
r2t-convert RAW OUT.r2t [--stats RUN.json] [--quiet]
```

- `RAW` is the raw dump directory; `OUT.r2t` is the processed trace directory. An existing `OUT.r2t` is replaced only if it is a processed trace.

- `--stats` gives the statistics file of the same run. The converter then compares the busy slot-cycles of every resource with the charges.
- Exit status 0: no check failed. Exit status 1: a check failed; the trace is written and the failures are on the standard error output. Exit status 2: the dump cannot be read (no manifest, an unknown layout, an unknown resource id).

4.2 Stages

Figure 3 shows the stages of the converter.

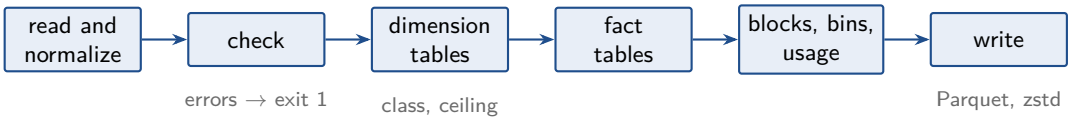


Figure 3: The stages of the converter.

1. **Read and normalize.** The converter reads the manifest and the record files (also zstd files) with the layouts of the manifest. It converts an older version to the current model: bridges become links, integer lanes become a capacity, lane-cycles become a usage, and the field `sub` becomes a message kind or a rate class by the role of the resource.
2. **Check** (Section 4.3).
3. **Derive the dimension tables**: nodes, PEs, timeline lanes, buses, attachments, links, memories, resources, rate classes and name tables, with the traffic class (Section 4.4) and the ceiling (Section 4.5) of every resource.
4. **Derive the fact tables**: tasks (with the label fields decoded), chunks, dependencies, programs, marks, phases and table entries, each sorted (Table 8).
5. **Build the time structures**: the charge blocks, the bins and the usage table (Section 4.6).
6. **Write** the Parquet tables with fixed encodings (Section 4.7), the manifest and the SQL views.

4.3 Checks

Check	Condition
references	every gid in chunks, dependencies, charges, programs and entries has a task record; every program occupant names a program
capacity	Rule V3.4, by a sweep over the charges of every resource with a rate
busy	with `--stats`: the slot-cycles of each resource equal its busy slot-cycles, within the rounding (the sum of *span*/65536)
classes	`sub` is a valid rate class or message kind
channels	a write channel has a read channel with the same beat, and $capacity = rate/beat_bytes$
topology	every resource id exists; every link has its direction resources
entries	$end \geq start$; the occupancy of a table never exceeds its capacity; an inbound invoke entry ends at its task's invoke; an outbound invoke entry starts at its task's `queued`; a VersionReset entry has no task
tasks	an untracked task has `version`, `queued` and `sent` null and is no producer; `dst_base` and `dst_len` are multiples of the granule, and `dst_len` is at least one granule

Table 6: The checks of the converter.

Rule V4.1 (relaxed checks). For a partial dump and for a dump without `summary.json`, the converter must not report a reference to a task without a record, and must not compare the busy slot-cycles. A windowed dump has no complete busy total.

4.4 Traffic class

The converter gives every bus and every fabric resource a traffic class. The class depends only on the link. Every transfer on a link has the class of the link.

1. A control bus and a control link are `control`.
2. Start at every bus that a `dram` memory attaches to, and follow the data links between buses. Do not pass through memories, PEs, Neutrinos or cache links. Every data bus that this search reaches is `global`.
3. Every other data bus is `local`.
4. A port, a shared-bus resource or a link direction has the class of its bus. A memory port is `global` for a `dram` memory and `local` otherwise. A PE resource is `compute`; a Neutrino scalar or completion resource is `scalar`.

Example. In the 2p device, `sbus`, every `clsbus`, every `smsbus` and the links between them are global; `lbus` is local; `cbus`, `clcbus` and `smcbus` are control.

4.5 Ceiling

The ceiling of a resource is the value that is 100 % on its utilization line.

- A resource with a rate: the rate. A rate class has its own rate as its ceiling.
- A port of a bus without a rate: the largest sum, over the run, of *amount*/*span* of the charges that are active at the same cycle. No view shows a value above it.

4.6 Time structures

A full forward pass has tens of millions of charges. The time structures let a reader load only what a time interval needs. Figure 4 shows them.

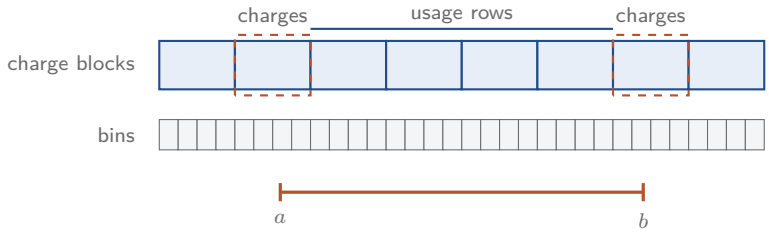


Figure 4: The time structures of a processed trace. A query over $[a, b)$ reads the usage rows of the blocks inside it and the charges of the two edge blocks.

Charge extent. The charge extent is the interval from the first charge start to the last charge end, clipped to the recorded window of a windowed dump.

Charge blocks. The charges table is sorted by block, then by resource, then by start. A **charge block** is a power-of-two span of cycles. Its size gives about 524288 charges per block on average over the charge extent, with a minimum of 65536 cycles. A charge belongs to the block of its start; the converter does not split a charge at a block end. Each block is one Parquet row group. The manifest lists for every block its start, its end and the largest charge end in it. A reader of the interval $[a, b)$ reads every block with $t_0 < b$ and $max_end > a$.

Bins. The bins table holds the busy slot-cycles and the amount of every resource, direction and rate class per bin. The bin width is the smallest power of two of at least 1024 cycles that gives at most 32768 bins over the charge extent. The first bin starts at the origin, the start of the charge extent rounded down to the width. A charge spreads its slot-cycles and its amount evenly over the bins that it covers. Only bins with a value are stored.

Usage table. The usage table sums the charges per resource, charge block and occupant: amount, slot-cycles, the number of charges, the first start and the last end. It is sorted by resource, so that the rows of one resource are contiguous.

Rule V4.2 (interval queries). The value of a resource over $[a, b)$ is the sum over the blocks fully inside $[a, b)$ from the usage table, plus the charges of the edge blocks, each clipped to $[a, b)$ in proportion to its overlap. At a cycle t, the occupants of a resource are its charges with $start \le t < start + span$.

4.7 Compression and encoding

Rule V4.3 (physical layout). The converter must write zstd pages (level 3) and must set the encodings explicitly: `DELTA_BINARY_PACKED` for integers, `BYTE_STREAM_SPLIT` for floating point values, `RLE_DICTIONARY` for strings and `PLAIN` for booleans. A list is a comma-separated text column.

Note. The interface has a fast decoder for exactly these encodings, and a slower general reader for other files. On the full forward pass, the delta encoding makes the fact tables 2.5 times smaller than the default dictionary encoding.

4.8 Size and time

Table	Rows	Size
tasks	1.49 M	25 MB
chunks	2.15 M	8 MB
dependencies	4.34 M	14 MB
charges	36.2 M	73 MB
bins	2.77 M	7 MB

Table 7: A processed trace of the full forward pass of pi0 on the 2p device (88.5 M cycles). The raw dump is 1.4 GB, or 226 MB with zstd. The conversion takes about 23 s.

5 Processed trace

A processed trace is a directory `NAME.r2t` with `manifest.json`, `views.sql` and one Parquet file per table. This section specifies processed trace version 1.8.

5.1 Manifest

The manifest of a processed trace has:

- `format` (`"r2t"`) and `version`;
- `run`: the run identity of the raw dump;
- `source`: the raw dump version, the converter commit, the filters, `complete` (false for a dump without `summary.json`) and the counts of references to tasks outside the trace;
- `end_time`, the label schema, the bins (`width`, `count`, `origin`) and the charge blocks;
- `tables`: every table and every column, with a description in plain words. A reader, a person or an agent, can learn the schema from the trace itself.

5.2 Tables

The dimension tables describe the device: `nodes` (with the capacity of each Neutrino table), `pes`, `lanes`, `buses`, `attachments`, `links`, `memories`, `memory_ranges`, `resources`, `rate_classes` and the name tables `opcodes`, `message_kinds`, `dirs`, `dep_states` and `table_names`.

The `resources` table gives, for every resource, its owner node, its kind, its unit, its rate, its capacity, its clock, its ceiling, its role fields, its traffic class, the flags `is_link` and `write`, and its run totals.

Table	Sort	Columns
`tasks`	gid	the record fields (`untracked` for bit 0 of `flags`), `lane`, the label fields
`chunks`	gid, index	`start`, `end`, `culprit`, `queue_culprit`
`deps`	consumer, index	`producer`, `state`, `remote`, `versioned`, `resolved`
`charges`	block, resource, start	`start`, `end`, `usage`, `cycles` (slot-cycles), `amount`, `dir`, `gid`, `program`, `msg`, `class`
`usage`	resource, block, occupant	`amount`, `cycles`, `n`, `first`, `last`
`bins`	resource, dir, class, bin	`busy`, `amount`
`programs`	nu, start	`id`, `name`, `spawn`, `start`, `end`
`entries`	nu, table, start	the record fields
`marks`, `phases`	start	the marks and the phases of the run

Table 8: The fact tables.

- Times are `INT64` SM cycles. A time that did not occur is NULL.
- The occupant of a charge is in two columns: `gid` for a task, `program` (an id in `programs`) for a program.
- The fact tables use small integer codes (direction, state, message kind, rate class). The name tables and the views give the names.

5.3 SQL access

```
r2t-sql OUT.r2t ["SELECT ..."]
```

`r2t-sql` runs DuckDB in the trace directory with `views.sql` loaded: without a query it is interactive, with a query it prints the result. In Python, `r2t.open_trace(path)` returns a DuckDB connection with the same views. The views are:

- one view per table over its Parquet file;
- `v_tasks` (with the timeline lane, the opcode name, the label name, and the durations $wait = ready - invoke$, $queue = issue - ready$, $exec = complete - issue$), `v_chunks`, `v_deps`, `v_charges` (with the slots held and the rate class name), `v_bins`, `v_entries`;
- the table functions `util(a, b)` (per resource: busy slot-cycles and amount over $[a, b)$, the busy fraction against the capacity and the utilization against the ceiling), `culprits(a, b)` (culprit cycles per resource and per subtree) and `active_tasks(a, b)`.

Example. The ten busiest links over an interval:

```
SELECT name, class, util FROM util(10000000, 12000000)
WHERE is_link ORDER BY util DESC LIMIT 10;
```

Example. The time of each task kind:

```
SELECT kind, count(*) AS n, sum(wait) AS wait, sum(queue) AS queue,
       sum(exec) AS exec
FROM v_tasks GROUP BY kind ORDER BY exec DESC;
```

Example. The bytes on an HBM port per task kind over the run, from the usage table:

```
SELECT t.kind, sum(u.amount) AS bytes
FROM usage u JOIN resources r ON r.id = u.resource
LEFT JOIN v_tasks t ON t.gid = u.gid
WHERE r.name = 'root.gpc1.hbm.port'
GROUP BY t.kind ORDER BY bytes DESC;
```

6 Views

The r2-vis interface shows a processed trace in three views: the block diagram, the timeline and the instant view. This section specifies what each view shows and how it reads the trace.

6.1 Architecture

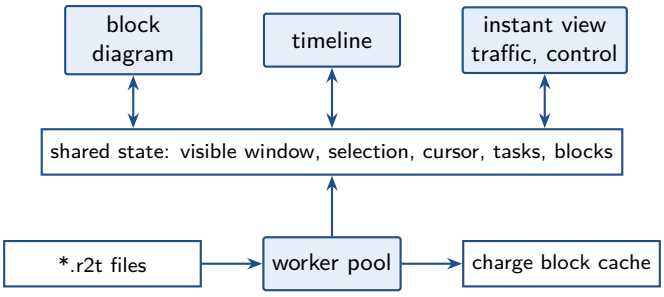


Figure 5: The interface. The views share one state; a pool of workers decodes the trace and answers the queries.

- The interface runs in a web browser. It reads the files of a processed trace from the local file system and keeps them in memory. It needs no server.
- A pool of workers decodes the Parquet files with a decoder for the encodings of Rule V4.3. One fixed worker answers the attribution queries (Section 6.5), so that it keeps its index of the tasks.
- At open, the interface reads the manifest, the dimension tables, the bins, the tasks, the chunks, the dependencies, the programs, the marks and the phases. It reads the charges by block, only for a visible interval narrower than about 2000 bins, and keeps the decoded blocks in a cache of least recently used blocks.

Note. The full forward pass of Table 7 opens in about 1 s. The views draw at 60 frames per second while the user pans and zooms.

6.2 Shared state

The views share one state:

- the *visible window*: the time interval on screen;
- the *selection*: a time interval that the user selects, or none;
- the *cursor*: one cycle that the user selects, or none. The selection and the cursor exclude each other;
- the selected rows and the selected tasks;
- the selected blocks: the nodes that the timeline shows.

Rule V6.1 (active time). A value of a view that is not a time series refers to the cursor if there is one, else to the selection if there is one, else to the visible window.

Rule V6.2 (recorded window). For a dump with a recorded window, every view must open on the recorded window, and every average must use only the part of its interval that is inside the recorded window. Outside the recorded window, a view must show “not recorded” and no value.

Input is the same in every view that has a time axis: a click selects the cursor, a drag selects an interval, ctrl and the mouse wheel (or a pinch) zoom, a horizontal scroll pans, the plain mouse wheel scrolls the rows, and a double click on the time axis returns to the first view. A plain click on a task selects it; shift and a click add or remove a task; a click on an empty place clears the task selection.

6.3 Block diagram

The block diagram shows the node hierarchy as nested boxes. Each box shows the programs of its Neutrino, the utilization of its tensor and vector resources, and the utilization of the global, local and control fabric that the node owns, over the visible window. A box that is too small shows its name only, or a color for its compute utilization. Each box has a control that adds its node to the timeline.

6.4 Timeline

The timeline has one group of rows per node, in the order of the hierarchy. A group has:

- a header with the programs of the Neutrino;

- one timeline lane per PE, and one for the tasks without a PE. A task bar spans the cycles from the issue of the task to the end of its last chunk. Tasks that overlap in time take separate rows of the lane;
- one row per resource, with its utilization line.

Utilization. A resource row shows the throughput: $amount/(ceiling \times \Delta t)$, where Δt is the time of one pixel. A dotted line marks the ceiling. The left label shows the value at the cursor, or the average over the selection or the visible window.

Groups of resources. A bus or a link is a node row over its ports and directions. A memory with banks or a write port is a node row over its banks and ports. A compute resource with rate classes is a node row over one row per rate class; each class row plots its throughput against its own rate. A node row is folded by default.

Rule V6.3 (folded rows). The line of a folded row is the maximum over its members of their utilization, per pixel. Its tooltip names the member that gives the maximum.

Level of detail. For a pixel wider than one bin, the interface reads the bins. It builds coarser levels by factor 4 and computes a node row from the members' values at the same level, so that a folded row is exact at a pixel that a level aligns with. For a pixel narrower than one bin, the interface reads the charges of the visible blocks and sums the overlapping charges.

Dependencies. When the user points at a task or selects it, arrows go from its producers to it and from it to its consumers. An arrow to a task that is not on screen ends at the edge of the plot with the name of its lane.

6.5 Attribution

Attribution connects resource usage to the tasks that cause it, in both directions. It reads the charges and the usage table with the queries of Rule V4.2.

Resource to tasks. For the selected resource row, lines go from the row, at the cursor or over the selection, to the occupants: a task to its bar, a program to its bar in the group header. The width of a line shows the share of the occupant. A setting limits the number of lines (32 by default). If the occupants exceed the limit, the interface groups them by lane and task kind, or by task kind only.

Tasks to resources. For the selected tasks, every resource row that a task charged shows the part of the line that the task causes: a band of height $amount/(ceiling \times span)$ over each charge. The bands of all occupants sum to the line. An arrow goes from the task to each such row, with the peak of the task's band as its label.

Control traffic. A setting, off by default, includes the charges of control resources and Neutrino work in the attribution. A Neutrino invokes a task long before the task runs, so these charges are far from the task in time.

6.6 Instant view

The instant view shows the device at the active time of Rule V6.1. It has the axis and the overview of the timeline at its top, and two parts: the traffic view and the control view.

Traffic view. The traffic view draws the fabric with the hierarchy from left to right: the blocks of the SMs, the buses of the SMs, the buses and blocks of the clusters, and the buses of the device.

- A bus is a vertical box. A link or a port is a strip between a block and a bus, or between two buses. A strip that passes behind a bus box without an attachment does not connect to it.
- The color of a strip is its traffic class. The strip shows the utilization and the throughput as text. A pattern moves along the strip at a speed that is proportional to the utilization. A link with traffic in both directions is two strips, one per direction.
- Strips that do not overlap in the horizontal direction share a row (interval packing).
- When the rows do not fit, adjacent nodes or blocks with the same name stem merge in pairs (`sm0-1`, `sm2-3`, then `sm0-3`). A merged box sums the strips that leave it, and shows the links inside it as loops. The zoom, a slider, or a click on a box selects the level of merging.

Control view. The control view draws the Neutrinos as a tree. The fill of a node shows the pressure of its tables, the occupancy divided by the capacity, for one table or for the worst or average table of a group. A panel shows the tables of the selected Neutrino with their occupancy and their entries.

6.7 Presentation

- The views show labels, values and legends. They show no explanatory text; a tooltip gives the detail of the item under the pointer.
- Values use one format: cycles with digit groups, counts with k and M, percentages with at most two significant digits, rates as `128 / 512 B/cycle`.
- A task name is its label, without the label fields that have one value in the whole trace, with its version if the name is ambiguous.
- The interface has a light theme and a dark theme.